c) Investigación:

➢ ¿Qué es Spring Cloud? ¿Cuáles son sus ventajas? ¿Qué mejoras aplica para la programación de microservicios?

Spring Cloud es un conjunto de herramientas del ecosistema Spring que se construye sobre Spring Boot y provee soluciones listas para usar a los problemas más comunes de los sistemas distribuidos, como la configuración centralizada, el descubrimiento de servicios, el enrutamiento, el balanceo de carga y la tolerancia a fallos. En lugar de que cada equipo desarrolle estas piezas desde cero, Spring Cloud las ofrece como librerías que se integran con una configuración mínima (anotaciones y propiedades), de modo que los desarrolladores se concentran en la lógica de negocio de cada microservicio.

Componentes principales:

- **Spring Cloud Config:** servidor de configuración centralizada y versionada. Permite que todos los microservicios obtengan sus propiedades desde un único lugar, según el ambiente (desarrollo, prueba, producción).
- **Service Discovery (Eureka / Consul):** los servicios se registran en un registro común y los demás los encuentran por nombre, sin tener que conocer su dirección física.
- **Spring Cloud Gateway:** API Gateway que funciona como único punto de entrada, enruta los pedidos al servicio correspondiente y permite aplicar seguridad y límites de uso.
- **Spring Cloud LoadBalancer:** reparte los pedidos entre las distintas instancias de un mismo servicio.
- **Circuit Breaker (Resilience4j):** corta las llamadas a un servicio que está fallando para evitar que el error se propague al resto del sistema.
- **Tracing y monitoreo (Micrometer / Zipkin):** permite seguir un pedido a través de varios servicios para detectar dónde falla o se demora.

Ventajas:

- **Menor tiempo de desarrollo:** resuelve de forma estándar problemas que se repiten en toda arquitectura de microservicios, evitando código repetitivo.
- **Integración con Spring Boot:** se adopta con facilidad en proyectos que ya usan Spring, aprovechando el mismo modelo de configuración y de dependencias.
- **Resiliencia:** incorpora mecanismos para que la caída de un servicio no derribe al resto de la aplicación.
- **Escalabilidad:** el descubrimiento y el balanceo de carga permiten agregar o quitar instancias sin modificar a los clientes.

- **Independencia de la infraestructura:** funciona tanto en servidores propios como en la nube, y existen integraciones con Kubernetes.

Mejoras que aplica a la programación de microservicios:

- **Configuración externalizada:** los cambios de configuración se aplican sin recompilar ni volver a desplegar cada servicio.
- **Comunicación entre servicios simplificada:** los servicios se invocan por su nombre lógico y no por una dirección fija, con balanceo de carga automático.
- **Tolerancia a fallos:** los reintentos y los circuit breakers hacen que las fallas parciales se manejen de manera controlada.
- **Punto de entrada único:** el gateway centraliza la autenticación, el enrutamiento y el control de tráfico, evitando repetirlos en cada servicio.
- **Observabilidad:** el seguimiento distribuido facilita diagnosticar problemas en sistemas donde un pedido atraviesa muchos servicios.